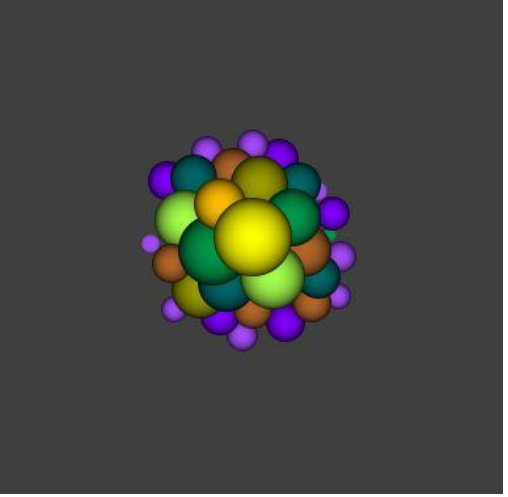
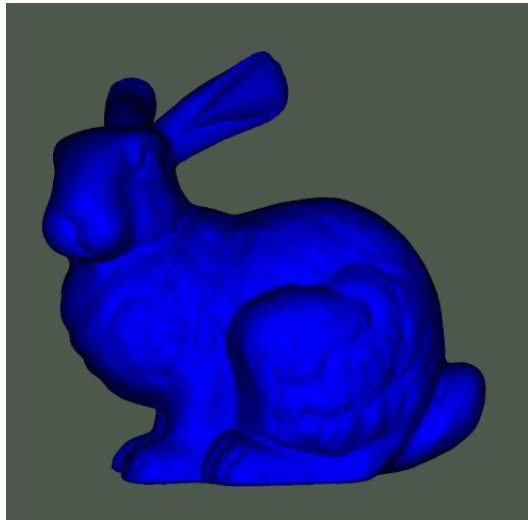


CS 440: Computer Graphics

Week 3

Lecture 6: WebGL & Affine Geometry



Acknowledgements: Cem Yuksel, University of Utah; Waqar Saleem, Habib University

Announcements and Recap

- Assignment 1 due 11:59pm, Friday
- Quiz next week

- Recap:
  - Imaging System: Synthetic Camera Model
  - Rasterization based Graphics Pipeline
  - Stages of Graphics Pipeline
  - WebGL Context

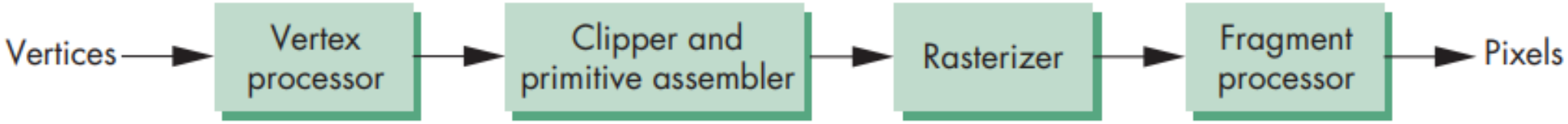
Today: WebGL & Affine Geometry

- WebGL Graphics Pipeline
- WebGL Functions
- Affine Space
- Geometric Primitives and Operations
- 3D Primitives

Anatomy of a WebGL Program

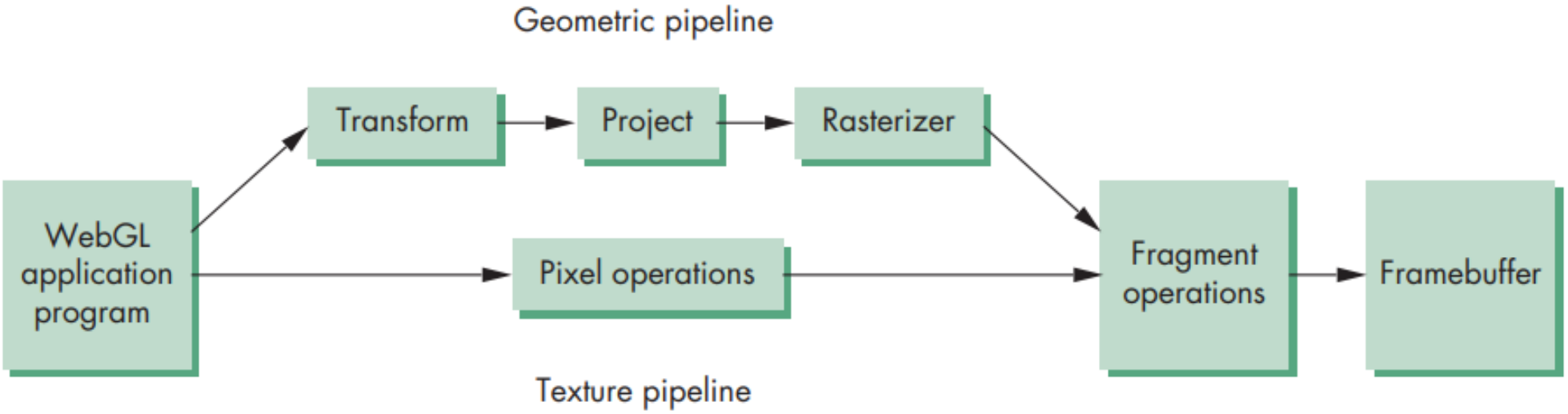
- Initialize the system
  - Connect with HTML Page
  - Specify shaders
- Compute the scene
  - Application logic
- Send scene data to GPU
  - Reserve GPU memory
  - Copy (vertex) data to GPU buffers
  - Bind GPU Data to shader variables
- Render scene
  - Trigger the pipeline

Recall: Graphics Pipeline



WebGL Pipeline

- Primitives and textures are dealt separately



Recall: Graphics API

- Programmer's View of a graphics system
- Independent of I/O details

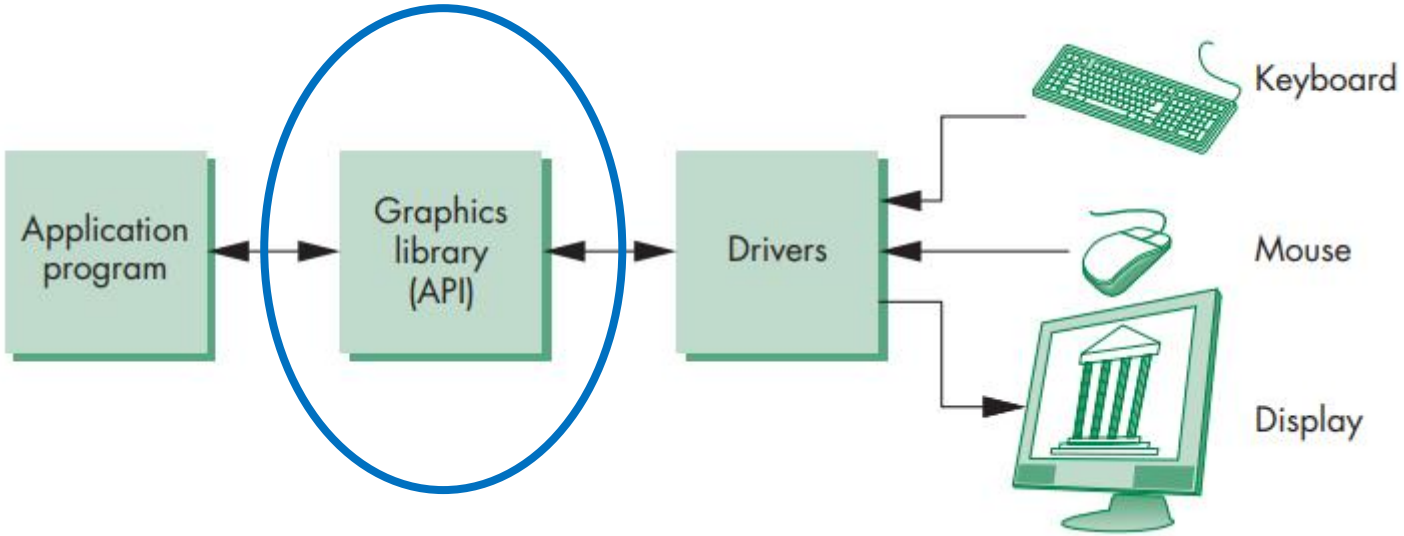
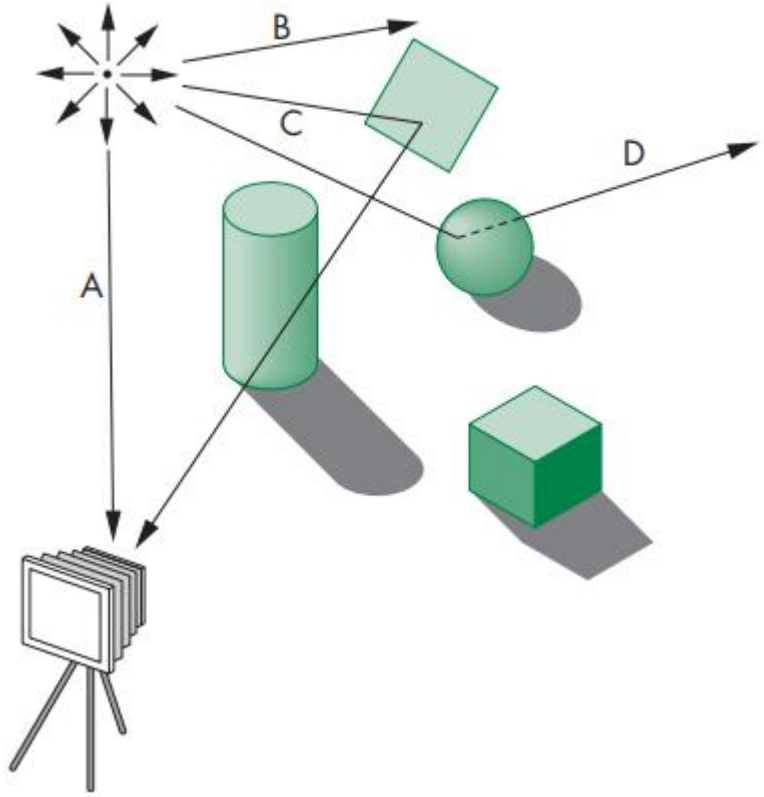


Image Formation and 3D Graphics API

- **Object**
  - Vertices and topology specified by the application program
- **Viewer**
  - Model view and projection matrices specified in the application program, applied through shaders
- **Light and Material**
  - Light and material properties specified in the application program, applied through shaders



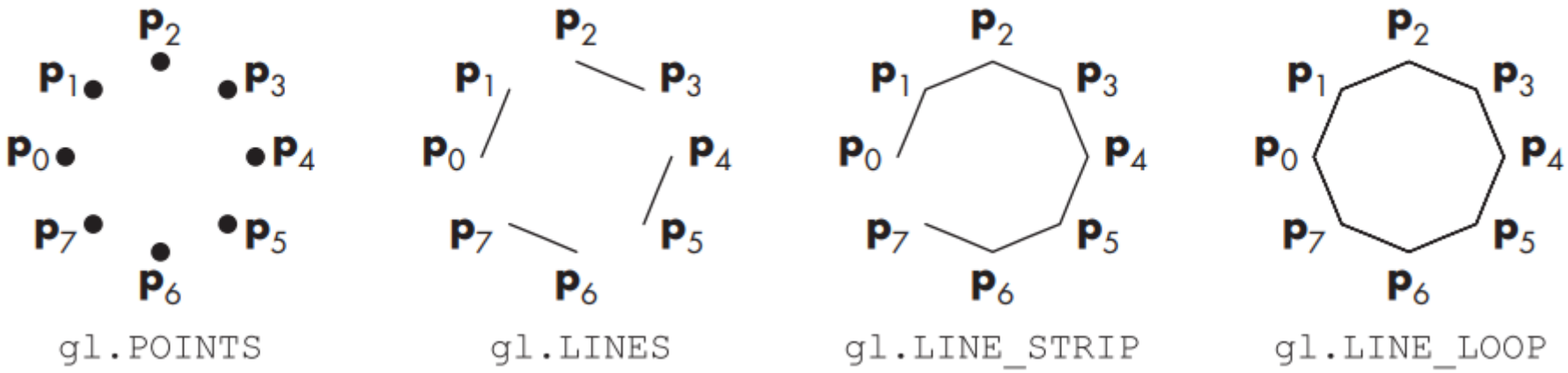
WebGL Functions

- Primitive functions
- Attribute functions
- Viewing functions
- Transformation functions
- Input functions
- Control functions
- Query functions

WebGL Functions

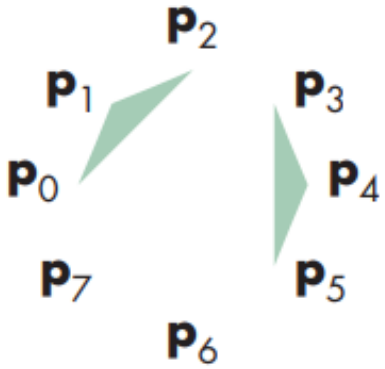
- **Primitive functions**
  - Primitives: Simplest geometric objects
  - Geometric primitives: Points, lines, triangles
  - Other primitives (such as polygons) can be approximated through the three geometric primitives
  - Raster primitives: Arrays of pixels (textures)

Point and Line Primitives in WebGL

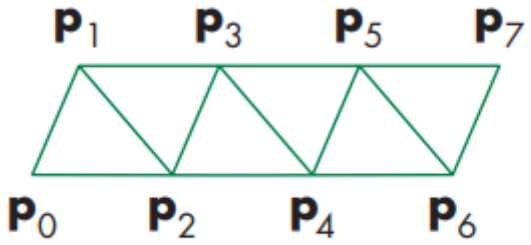


Polygon Primitives in WebGL

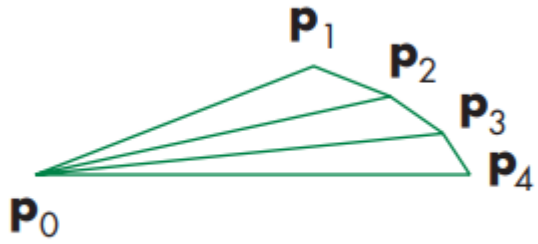
- Polygons must be **simple, convex** and **flat**
- In 3D, the triangle and the plane on which it lies must be **non-collinear**
- Triangles are the only polygon primitives in WebGL



gl.TRIANGLES



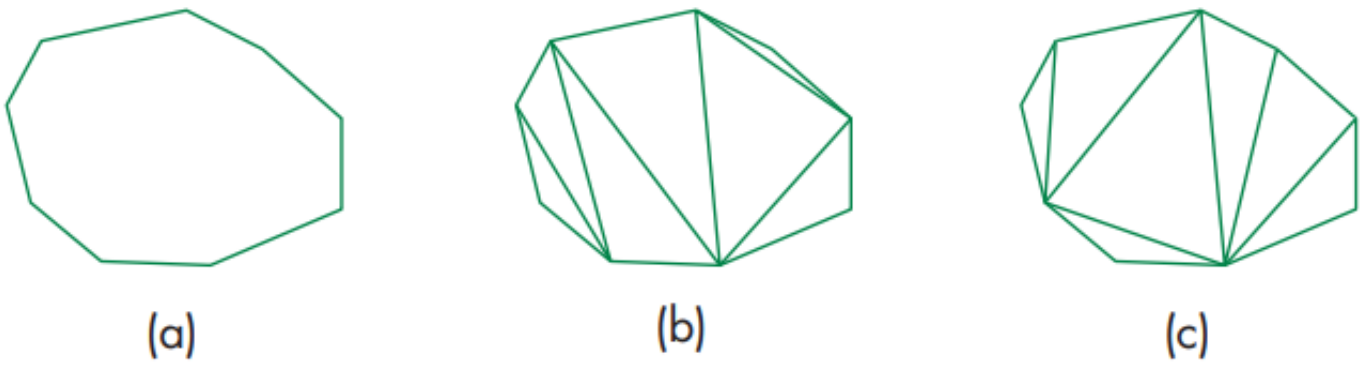
gl.TRIANGLE_STRIP



gl.TRIANGLE_FAN

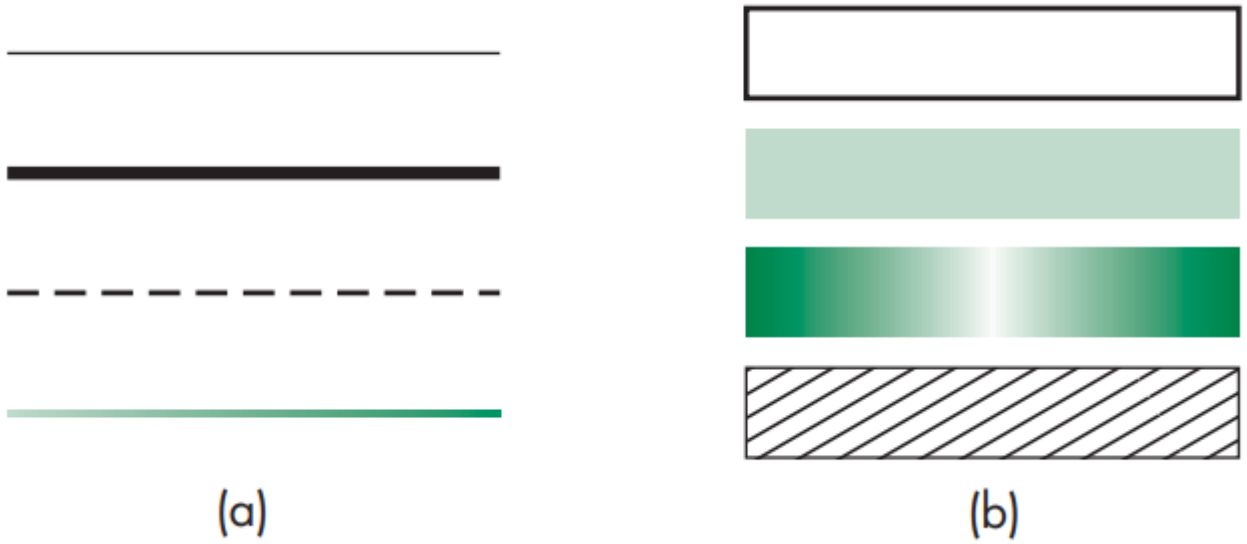
Triangulation

- All polygons are broken down into a set of triangles
- Different triangulations for same shape exists
- **Tessellation:** Breaking down a polygon into smaller polygons (not necessarily triangles)



WebGL Functions

- **Attribute functions**
  - How the primitives are displayed
  - Color of primitives
  - Vertex attributes



Lines

Polygons

WebGL Functions

- **Viewing functions**
  - Perspective or orthographic views
  - WebGL does not have viewing functions
  - Transformations are applied to get the desired view in WebGL

- **Transformation functions**
  - Rotation, scaling, translation
  - WebGL performs transformations using matrices either in the application program or shaders

WebGL Functions

- **Input functions**
  - Interaction of WebGL with user
- **Control functions**
  - Communication with windows system
  - Application program initialization and execution
- **Query functions**
  - Accessing other information within API

Affine Geometry

#MyCampusClick | Dawood Habib Yohsin Hall, Library

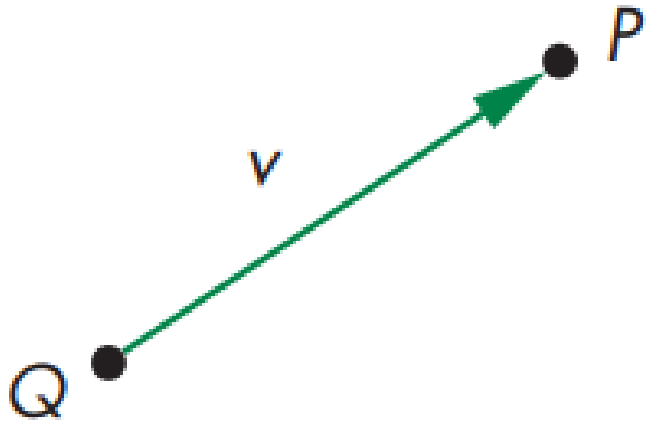
Recall: Scalars, Vectors, Points

- **Scalar:** Members of a set that follow certain rules
  - E.g. Integers, real or complex numbers
  - No geometric property
  - Operations: Addition, multiplication
  - Commutative and associative
  - Inverse exists for each operation
- **Point:** Position in space
  - Has no size or shape
- **Vector:** Has direction and magnitude
  - Does not have a fixed position in space
  - E.g. Velocity, force, directed line segment
  - Operations: Addition, inverse, dot product etc.

Recall: Point-Vector Operations

- Points can be added with vectors to displace a point

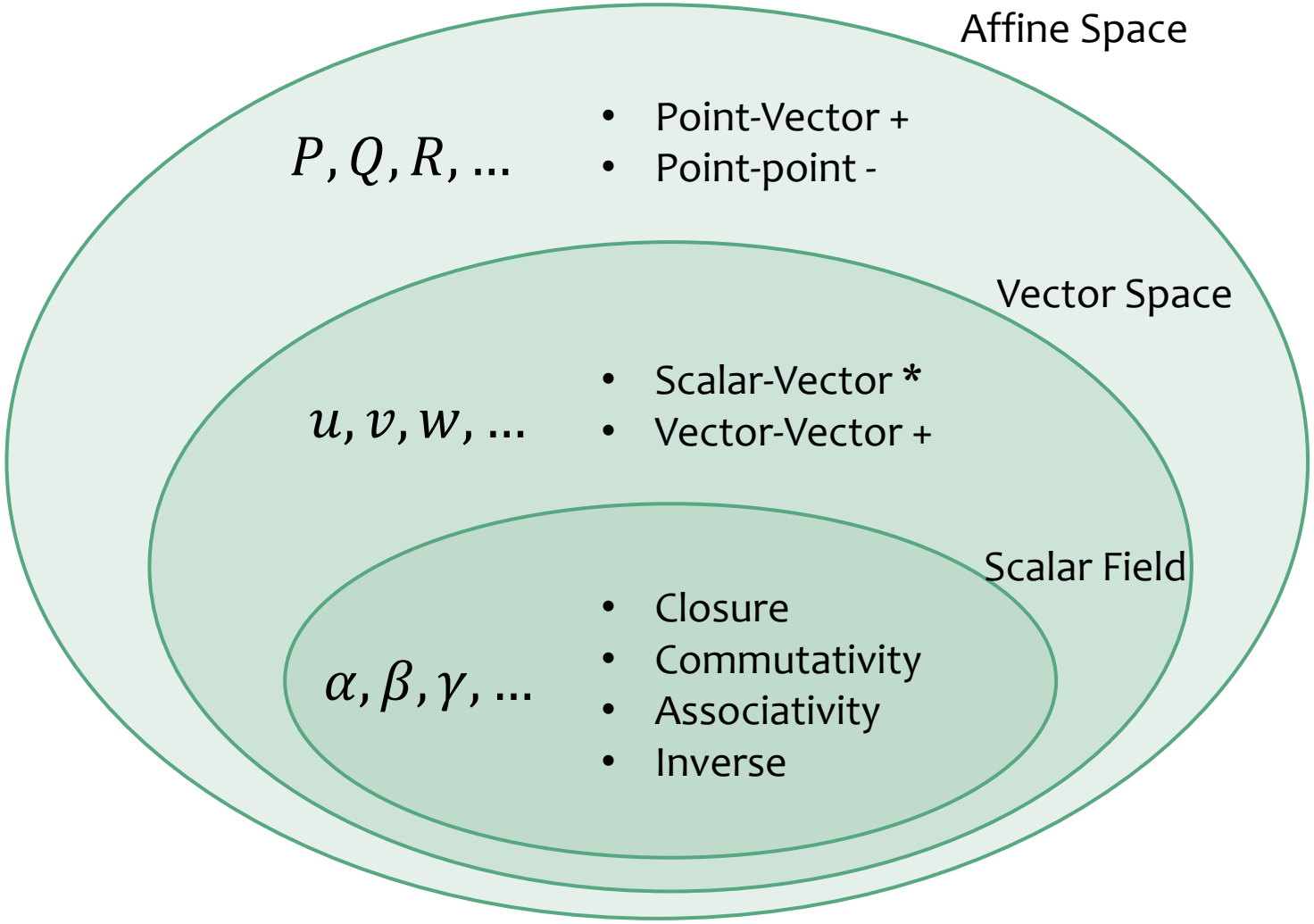
$$P = Q + v$$



- Point-point subtraction yields a vector

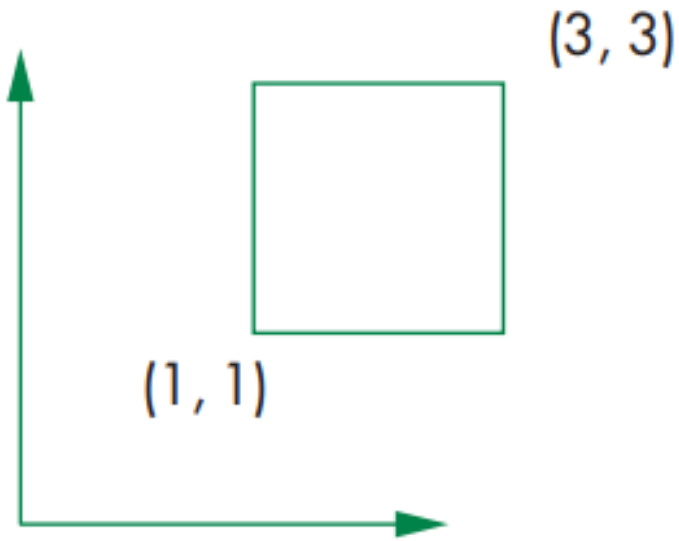
$$v = P - Q$$

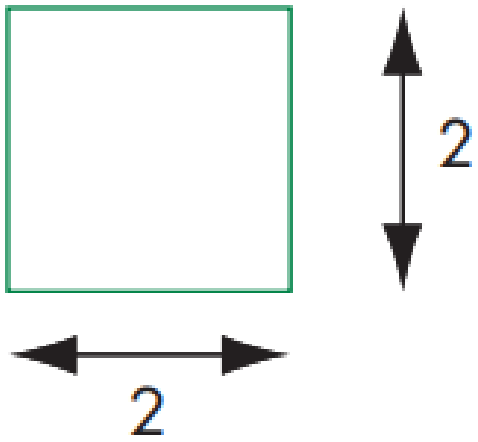
Mathematical Spaces



Affine/Coordinate Free Geometry

- Without origin and axis, we cannot specify points
- Geometric relationships are preserved

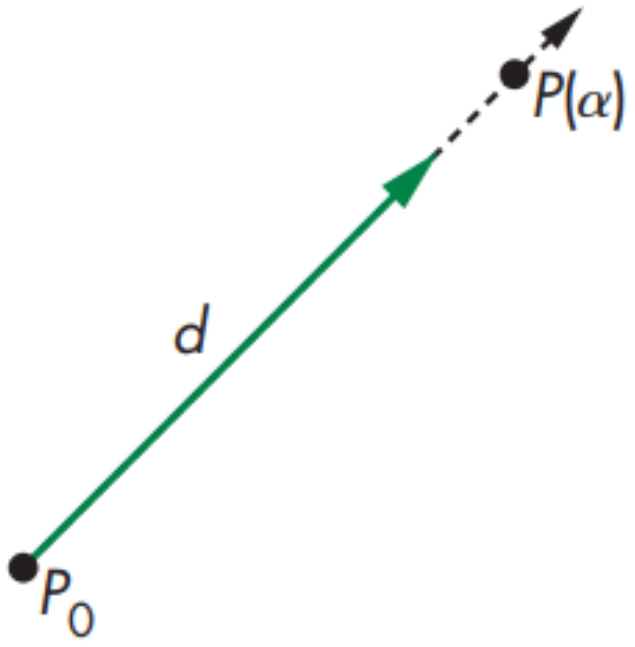




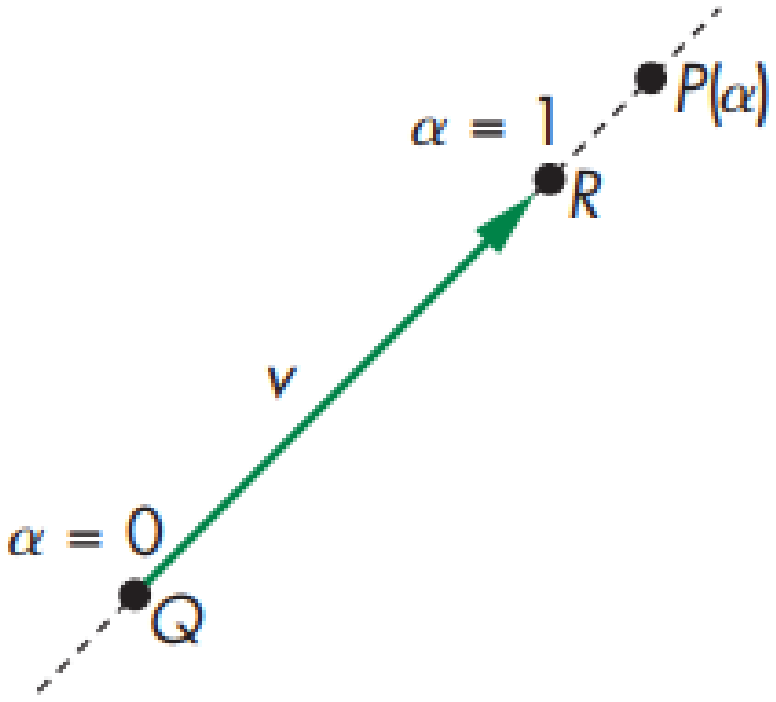
Geometric Primitives: Line

- Parametric form of a line
- Sum of point and vector is a **line**

$$P(\alpha) = P_0 + \alpha d$$



Affine Sum

- Point-point addition does not exist
- Affine sum is always equivalent to 1

$$P = \alpha R + (1 - \alpha)Q$$